Revised Report

Group 05
Dimitrios Moutaftsdis 9920 , moutdimi@ece.auth.gr
Gaitanidis Neoklitos 10031
Nikolas Stamatis 9574

02 June 2025

1 Introduction

This report attempts to compensate for earlier shortcomings. This is our best attempt yet to comply with assignment requirements. We will try to loyally reproduce the results shown in the Paper as well as provide pseudocode examples for core algorithm functionality.

2 Quick Start:

The code needs the contents of the folders titled Strategies , Helper Functions and Core Functions to be runnable. In order to perform quick checkup run the contents of Tests. To reproduce the experiments corresponding to the figures shown here check the corresponding Examples folder. We advise downloading all the files in the Repository and dumping them in the same folder, after extracting them from the individual ones. Adding each file to path should make them run. Follow this link to find the repo: https://github.com/moutdimi/Game-Theory-Group-05

3 Global Functions in Pseudocode

The basic idea behind our algorithms is to return 1 in case of cooperation and 2 in case of defection so that we can index the payoff matrix accordingly. The move history is initialized as a NaN table and filled as the game progresses.

3.1 All C

3.1.1 Goal:

Always Cooperate.

3.1.2 Pseudocode:

MOVE = 1

RETURN MOVE

3.2 All D

3.2.1 Goal:

Always Defect.

3.2.2 Pseudocode:

MOVE = 2

RETURN MOVE

3.3 Tit For Tat

3.3.1 Goal:

Start by Cooperating. Then imitate opponents last move.

3.3.2 Pseudocode:

IF FIRST MOVE:

RETURN MOVE = 1

FIND MOST RECENT OPPONENT MOVE

IF 1:

RETURN MOVE = 1

ELSE IF 2:

RETURN MOVE = 2

3.4 Periodic CD

3.4.1 Goal:

Start by cooperating ,then defect. Repeat regardless.

3.4.2 Pseudocode:

IF FIRST MOVE:

RETURN MOVE = 1

FIND LAST OWN MOVE

IF 1:

RETURN MOVE = 2

ELSE IF 2:

RETURN MOVE = 1

3.5 Periodic D D C

3.5.1 Goal:

Start by Defecting twice, then Cooperating once. Repeat regardless.

3.5.2 Pseudocode:

IF FIRST MOVE:

RETURN MOVE = 2

IF SECOND MOVE:

RETURN MOVE = 2

IF THIRD MOVE:

RETURN MOVE = 1

FIND LAST , SECOND-TO-LAST OWN MOVES

```
IF LAST IS 1:
    RETURN MOVE = 2
ELSE IF LAST IS 2 AND SECOND-TO-LAST IS 1:
    RETURN MOVE = 2
ELSE IF LAST IS 2 AND SECOND-TO-LAST IS 2:
    RETURN MOVE = 1
```

3.6 Periodic C C D

3.6.1 Goal:

Start by Cooperating twice, then defecting once. Repeat regardless.

3.6.2 Pseudocode:

```
IF FIRST MOVE:
    RETURN MOVE = 1
IF SECOND MOVE:
    RETURN MOVE = 1
IF THIRD MOVE:
    RETURN MOVE - 2
FIND LAST , SECOND-TO-LAST OWN MOVES
IF LAST IS 2:
    RETURN MOVE = 1
ELSE IF LAST IS 1 AND SECOND-TO-LAST IS 2:
    RETURN MOVE = 2
ELSE IF LAST IS 1 AND SECOND-TO-LAST IS 1:
    RETURN MOVE = 2
```

3.7 Periodic C C C C D

3.7.1 Goal:

Start with four consecutive Cooperations, then defect once. Repeat regardless.

3.7.2 Pseudocode:

```
IF FIRST MOVE:
    RETURN MOVE = 1
IF SECOND MOVE:
```

```
    RETURN MOVE = 1

IF THIRD MOVE:

    RETURN MOVE = 1

IF FOURTH MOVE:

    RETURN MOVE = 1

IF FIFTH MOVE:

    RETURN MOVE = 2

FIND FOUR LAST OWN MOVES

IF LAST IS 2:

    RETURN MOVE = 1

ELSE IF LAST IS 2 AND SECOND-TO-LAST IS 1 AND THIRD-TO-LAST IS 1 AND FOURTH-TO-LAST IS 1:

    RETURN MOVE = 2

ELSE:

    RETURN MOVE = 1
```

3.8 Prober

3.8.1 Goal:

Start by playing Defection, then Cooperation twice. If the opponent doesn't defect in moves 2 or 3, Defect every time onward. If a defection was picked up, play Tit For Tat starting from move 4. Give a chance for Cooperation after mutual defection.

3.8.2 Pseudocode:

```
IF FIRST MOVE:

    RETURN MOVE = 2

IF SECOND MOVE:

    RETURN MOVE = 1

IF THIRD MOVE:

    RETURN MOVE = 1

IF OPPONENT MOVES 2 AND 3 ARE 1:

    RETURN MOVE = 2

FIND LAST OWN , OPPONENT MOVES

IF LAST OWN AND LAST OPPONENT MOVES ARE 2:

    RETURN MOVE = 1

IF LAST OPPONENT MOVE IS 1:
```

MOVE = 1

ELSE:

MOVE = 2

MATLAB RETURNS MOVE

3.9 Soft Majority

3.9.1 Goal:

Start by Cooperating. Then play the most common opponent move. Cooperate in case of a tie.

3.9.2 Pseudocode:

IF FIRST MOVE:

RETURN MOVE = 1

EXTRACT OPPONENT MOVES IN VECTOR

SUBTRACT 1 (*Thus voting occurs through summation*

IF MAJORITY MOVE OR TIE IS 0:

MOVE = 1

IF MAJORITY MOVE IS 1:

MOVE = 2

MATLAB RETURNS MOVE

3.10 Gradual

3.10.1 Goal:

Start by Cooperating. Upon detecting opponent Defection, retaliate accordingly. Then Cooperate twice.

3.10.2 Pseudocode:

IF FIRST MOVE:

MOVE = 1

RETALIATIONS = 0

DEFECTIONS = 0

COOPERATIONS = 0

RETURN ALL

IF RETALIATIONS ARE POSITIVE:

MOVE = 2

RETALIATIONS -= 1

RETURN ALL

IF COOPERATIONS ARE POSITIVE:

MOVE = 1
COOPERATIONS -= 1

RETURN ALL

FIND LAST OPPONENT MOVE

IF LAST MOVE IS 2:

DEFECTIONS += 1

RETALIATIONS = DEFECTIONS

COOPERATIONS = 2

MOVE = 2

RETALIATIONS -= 1
RETURN ALL

MOVE = 1

MATLAB RETURNS ALL

Now we are done when it comes to startegies. Let's examine the functions used to simulate moves, rounds and matches.

3.11 Move 2

3.11.1 Goal:

Accepts a single player strategy and a full move history tables as handles. It also carries certain flags used by Gradual. It only plays the row player. Meaning that the move history must be passed in a way where the second column refers to the column (the "other") player. It calls the appropriate function based on the specified strategy. The flags are returned as well.

3.11.2 Pseudocode:

IF PLAYER PLAYS STRATEGY X:

MOVE AND HANDLES = FUNCTION-X(MOVE HISTORY , HANDLES)

MATLAB RETURNS ALL

3.12 Match

3.12.1 Goal:

This function pits two opponent against each other. It initializes an empty move history table of the appropriate size, as longs as some signal flags. Then iterates over the game length and calls the Move function to obtain row and column player round results. Then it stores results and updates the move history. The distinction between row and column player is achieved by flipping the move history table before passing it to Move. This way, column 2 always corresponds to the column ("other") player.

3.12.2 Psaudocode:

```
INITIALIZE MOVE HISTORY

INITIALIZE SIGNAL FLAGS

INITIALIZE MATCH RESULT AS 0

FOR GAME LENGTH ITERATIONS:

    GET ROW PLAYER MOVE AND UPDATE FLAGS

    GET COLUMN PLAYER MOVE AND UPDATE FLAGS

    UPDATE MOVE HISTORY

    DETERMINE ROUND RESULT

    UPDATE MATCH RESULT

MATLAB RETURNS MATCH RESULT
```

3.13 Axel 2

3.13.1 Goal:

This function is responsible for simulating a Round Robin-Axelrod Tournament of a mixed population of players paying different strategies. It accepts as handles the payoff matrix , population composition and game length. Iterates through all player vs player combinations and stores overall performance in a vector that contains each individual's strategy and total payoff.

3.13.2 Pseudocode:

```
GET TOTAL POPULATION

INITIALIZE PLAYER VECTOR

FOR TOTAL POPULATION - 1 ITERATIONS:

    FOR REMAINING PLAYERS BEYOND CURRENT INDEX ITERATIONS:

        EXTRACT EACH PLAYER'S STRATEGY BY INDEXING PLAYERS VECTOR

        OBTAIN MATCH RESULT BY CALLING MATCH2(...)

        STORE MATCH RESULTS BY UPDATING CURRENT POSITION OF PLAYERS VECTOR

        NUMBER OF MATCHES += 1

MATLAB RETURNS ALL
```

The above three functions will become our tools to simulate the Evolutionary Tournaments.

4 Fitness Dynamics

The first part of the assignment requires adopting the Fitness Evolution doctrine as shown in the Paper. We did discuss this in the previous report, however we will revisit it once again to produce a more complete result.

4.1 Theoretical Tournament Fitness Dynamics

4.1.1 Description:

This function sets out to perform the experiments exactly as shown in the Paper. Evolutionary Tournaments are about modifying the population composition based on certain criteria. In this implementation, the dominance of each group is determined by their Fitness. In other words, the percentage of the total payoff they managed to accumulate. The largest earners will multiply. Here we must note that as the Tournament progresses, the dominance of each strategy doesn't remain fixed. On the other hand, since it is extremely sensitive to the population composition we observe different phenomena of convergence, reducing, increasing or periodic oscillatory behavior etc. Some strategies might be strong in a particular setting, but slight modification in population composition might bring them in a disadvantage and vice versa. For the Theoretical part, we faithfully adopt the following update rule:

$$
\begin{aligned}
g_n(A) &= W_n(A)\,V\big(A \mid A\big) + W_n(B)\,V\big(A \mid B\big) + W_n(C)\,V\big(A \mid C\big) - V\big(A \mid A\big),\\
g_n(B) &= W_n(A)\,V\big(B \mid A\big) + W_n(B)\,V\big(B \mid B\big) + W_n(C)\,V\big(B \mid C\big) - V\big(B \mid B\big),\\
g_n(C) &= W_n(A)\,V\big(C \mid A\big) + W_n(B)\,V\big(C \mid B\big) + W_n(C)\,V\big(C \mid C\big) - V\big(C \mid C\big).\\
t(n) &= W_n(A)\,g_n(A) \;+\; W_n(B)\,g_n(B) \;+\; W_n(C)\,g_n(C).\\
W_{n+1}(A) &= \frac{\Pi\, W_n(A)\,g_n(A)}{t(n)},\\
W_{n+1}(B) &= \frac{\Pi\, W_n(B)\,g_n(B)}{t(n)},\\
W_{n+1}(C) &= \frac{\Pi\, W_n(C)\,g_n(C)}{t(n)}.
\end{aligned}
\tag{1}
$$

In other words, the size of each Strategy Group in the next Generation corresponds to the fraction of the total payoff it accumulated multiplied by the population size with floor division integer rounding. Thus we understand that the larger the score hoarding for a given group, the largest its representation in the next generation. Our function accepts as handles the payoff table, the set of different strategies and their initial population as well as the game and tournament lengths. It returns the population progression, each generations best strategy and each group's fitness in each generation. To skip the player per player simulation we define a matrix that contains the payoff of strategy vs strategy match-ups. We do that by iterating through all active strategy combinations and using the Match function to extract total payoff for the given game length.

4.1.2 Pseudocode TourTheFit:

EXTRACT NUMBER OF STRATEGIES WITH NON ZERO POPULATION

CALCULATE TOTAL POPULATION

INITIALIZE POPULATION PROGRESSION , GENERATIONAL FITNESS AND BEST STRATEGY MATRICES

INITIALIZE THE STRATEGY VS STRATEGY PAYOFF MATRIX

FOR NUMBER OF ACTIVE STRATEGY ITERATIONS:

    FOR NUMBER OF ACTIVE STRATEGY ITERATIONS:

        EXTRACT THE RESULT OF CURRENT FACE-OFF BY CALLING MATCH2(...)

        STORE IT IN i,j MATRIX ELEMENT

FOR NUMBER OF GENERATIONS ITERATIONS:

    EXTRACT CURRENT POPULATION COMPOSITION W

    CALCULATE g AS DESCRIBED IN THE FORMULA

    CALCULATE TOTAL ACCUMULATED SCORE

STORE EACH STRATEGY'S FITNESS IN fit VECTOR

CALCULATE THE POPULATION COMPOSITION FOR THE NEXT GENERATION BY USING ELEMENT-WISE OPERATIONS FOR THE W AND g VECTORS TO MATCH THE FORMULA

STORE IT IN pop VECTOR

STORE THE BEST PERFORMING STRATEGY IN bst

IF TWO ARE TIED:

STORE THEM BOTH IN bst

MATLAB RETURNS ALL

4.2 Player-to-Player Simulation Tournament Fitness Dynamics

The core idea remains the same. Evolution is still determined by sub-group fitness. However, a certain problem arises: simply rounding down would most likely reduce the total population. Thus a heuristic rule for supplementing is required. We decided to use the Largest Remainder (or Hamilton's) rule. The group with the largest decimal remainder is compensated based on the population deficit. This way, we attempt to make up for "unjust" rounding. For player-to-player simulation we use the Axel function, which does exactly that. We run it for the required number of generations. We then iterate through the participating strategies to extract a mask that corresponds to all the players playing strategy X. Then we get the total strategy payoff and determine its fitness by floor rounding. A deficit is then calculated. Based on the above rule we compensate the corresponding "tribe". Then the next generation begins.

4.2.1 Pseudocode TourSimFit:

EXTRACT NUMBER OF ACTIVE STRATEGIES

CALCULATE INITIAL POPULATION

INITIALIZE POPULATION , FITNESS AND BEST STRATEGY PER GENERATION MATRICES

FOR NUMBER OF GENERATIONS ITERATIONS:

EXTRACT SIMULATION RESULTS THROUGH AXEL

INITIALIZE TOTAL PAYOFF VECTOR

FOR NUMBER OF STRATEGY ITERATIONS:

EXTRACT INDEX MASK OF PLAYERS PLAYING STRATEGY i

APPLY MASK TO SCORES VECTOR AND SUM

STORE SUM TO FITNESS VECTOR

CALCULATE GROUP PLAYER COUNTS

ROUND DOWN TO GET NEW POPULATION

CALCULATE DEFICIT

IF DEFICIT IS NON-ZERO:

FIND THE GROUP WITH THE LARGEST ROUNDING ERROR

COMPENSATE BY ADDING DEFICIT NUM OF INDIVIDUALS TO THE RESPECTIVE POSITION

IN POPULATION VECTOR

STORE COMPOSITION IN POP TABLE

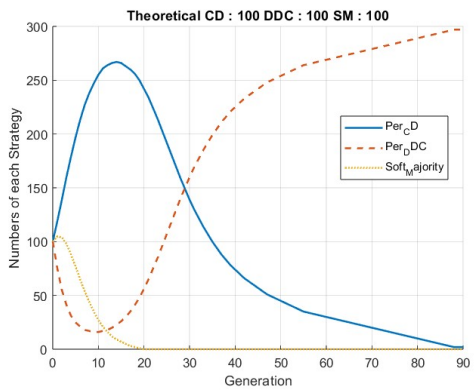
FIND BEST STRATEGY

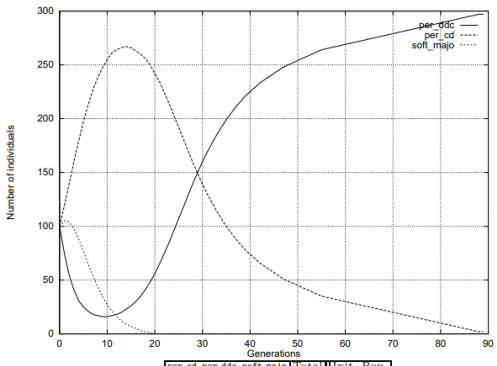
STORE IN BST VECTOR

MATLAB RETURNS ALL

4.3 Fitness Dynamics Result Comparison

In this segment we shall present and discuss the results shown in the Paper and compare them to both the ones produced by both the Theoretical and Simulation approaches. Each case will be presented side by side to showcase the differences in dynamics. To generate each figure set, a script titled Fit Fig X needs to be executed to produce the corresponding figure number. The Simulation part is not included. It is found on each script but its commented out. To reproduce the results of each setting at least three hours per run are required. We will perform separate, shorter experiments.
Here we compare TourTheFit with the Paper's results. In all cases there is an exact match. Only a very slight numerical deviation is observed when Gradual is involved. Most likely our implementation differs slightly, however we weren't able to locate how. The overall trend remains the same, however.



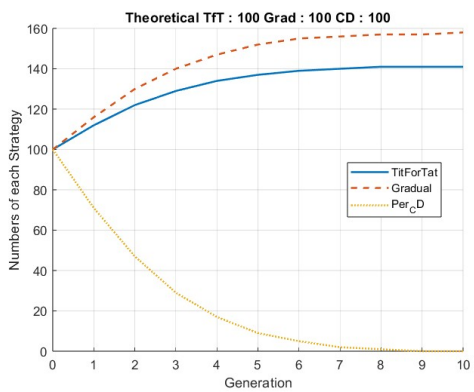


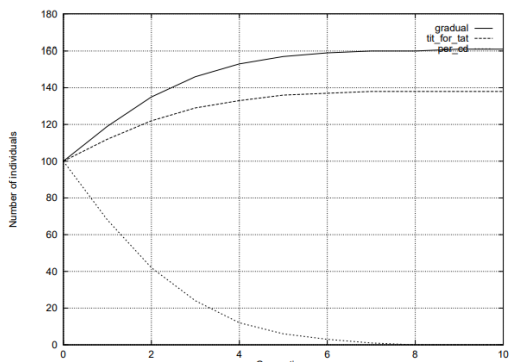
	per_cd	per_ddc	soft_majo	Total	Init. Pop.
per_cd	2000	1665	4000	7665	100
per_ddc	2500	1666	671	4837	100
soft_majo	1500	2331	3000	6831	100

Fig. 1. Defectors may be strong

Figure 1

Defectors may be strong.



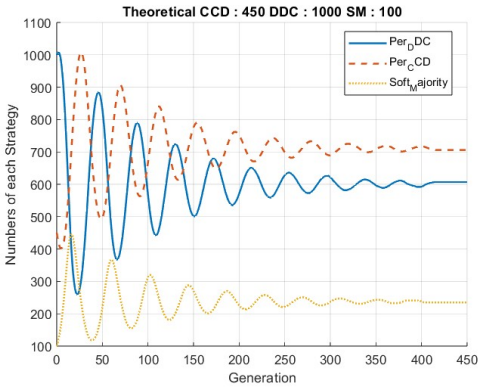


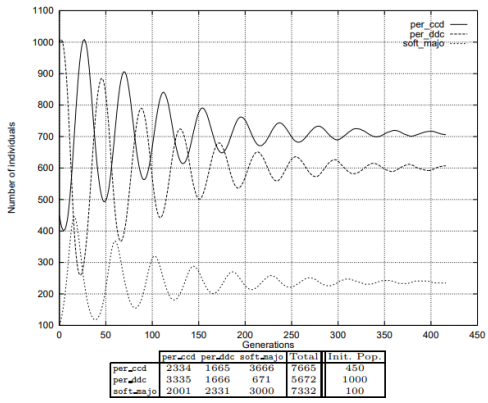
	tit_for_tat	gradual	per_cd	Total	Init. Pop.
tit_for_tat	3000	3000	2498	8498	100
gradual	3000	3000	2955	8955	100
per_cd	2503	615	2000	5115	100

Fig. 2. Monotonous convergence

Figure 2

Monotonous Convergence.



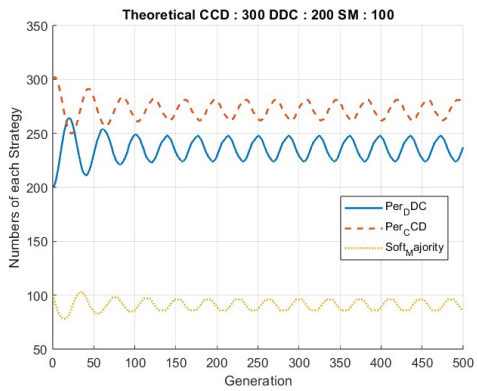


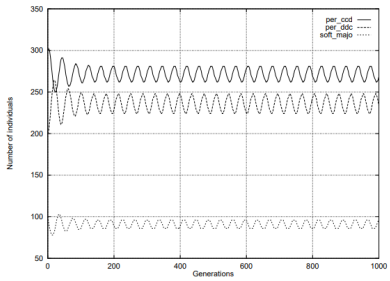
	per_ccd	per_ddc	soft_majo	Total	Init. Pop.
per_ccd	2334	1665	3666	7665	450
per_ddc	3335	1666	671	5672	1000
soft_majo	2001	2331	3000	7332	100

Fig. 3. Attenuated oscillatory movements

Figure 3

Attenuated oscillatory movements.



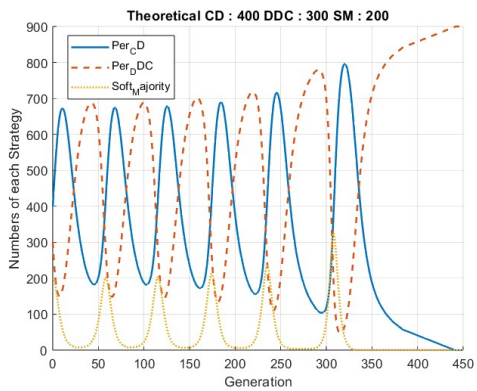


	Init. Pop.
per_ccd	300
per_ddc	200
soft_majo	100

Fig. 4. Periodic movements

Figure 4

Periodic movements.



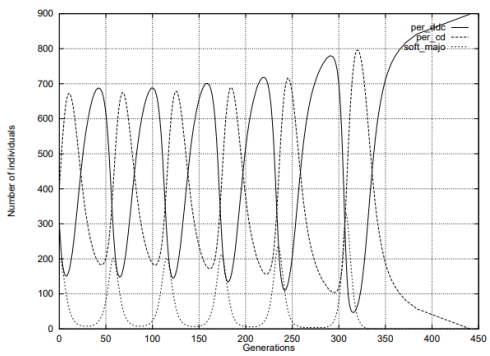
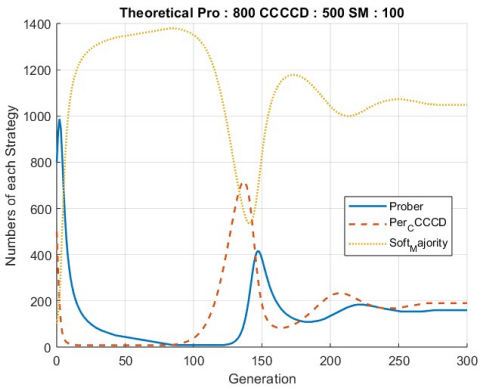
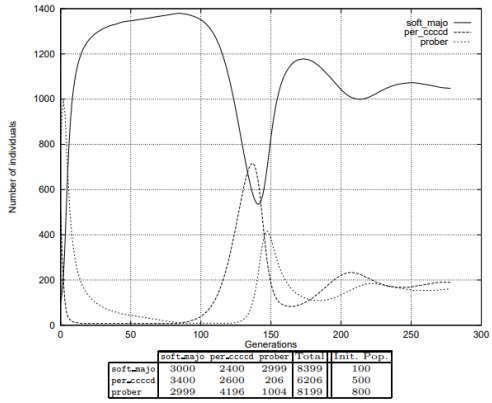


Fig. 5. Increasing oscillations

Figure 5

Increasing oscillations.



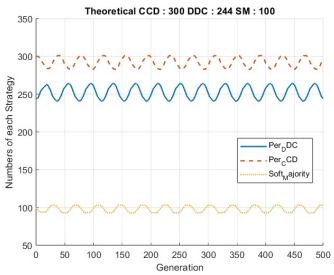


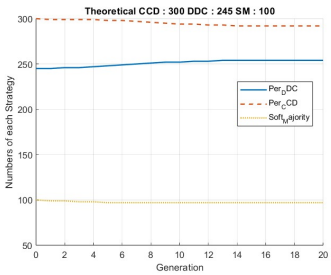
	soft_majo	per_ccccd	prober	Total	Init. Pop.
soft_majo	3000	2400	2999	8399	100
per_ccccd	3400	2600	206	6206	500
prober	2999	4196	1004	8199	800

Fig. 6. Disordered oscillations

Figure 6

Disordered oscillations.





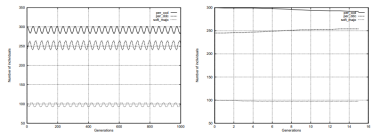
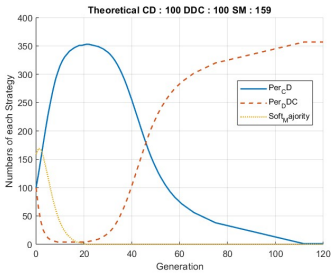
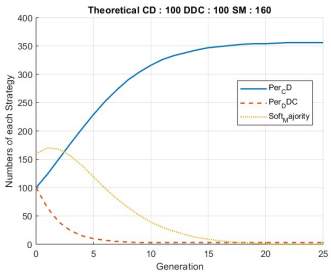


Fig. 7. Sensitivity of dynamics to population's size. All parameters are identical except for the initial size of `per_ddc` which is 244 on the left and 245 on the right

Figure 7

Sensitivity of dynamics to population's size.





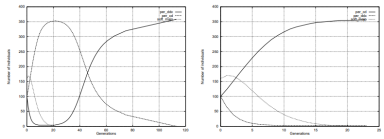
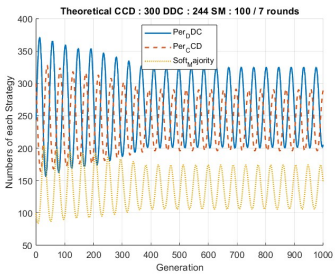
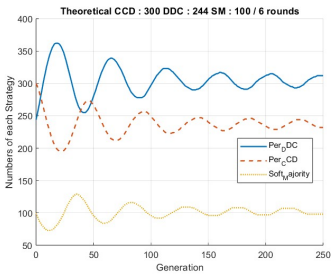


Fig. 8. Sensitivity of winner to population's size. All parameters are identical except for the initial size of `soft_majo` which is 159 on the left and 160 on the right.

Figure 8

Sensitivity of winner to population's size.





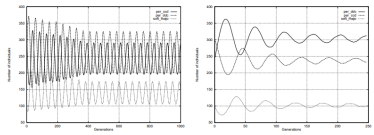
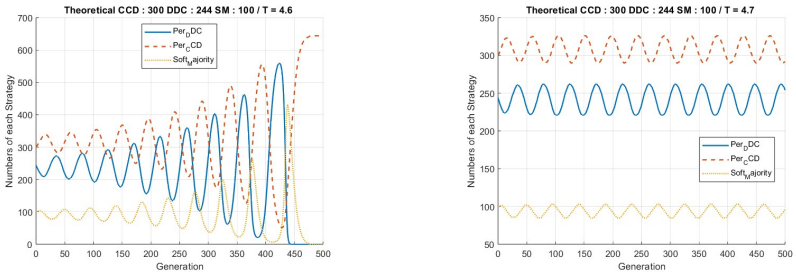


Fig. 9. Sensitivity to game length. All parameters are identical except for the game length which is 7 moves on the left and 6 on the right.

Figure 9

Sensitivity to game length.



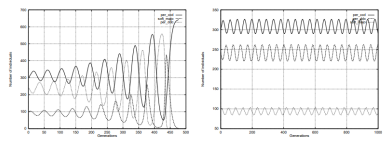
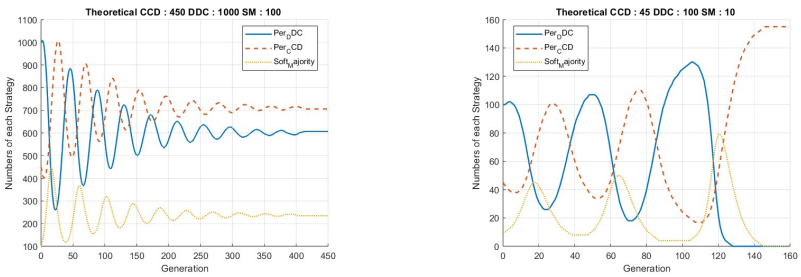


Fig. 10. Sensitivity to CIPD payoff. All parameters are identical except that T=4.6 on the left and T=4.7 on the right.

Figure 10

Sensitivity to CIPD payoff.



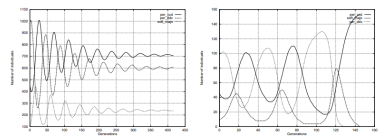


Fig. 12. Sensitivity to repartition computation method. All parameters are identical except that populations on the right are divided by 10.

Figure 12

Sensitivity to repartition computation method.

5 Imitation Dynamics

In this section we will discuss Imitation Dynamics in both a theoretical and simulation context. It is a less passive evolutionary approach. A predetermined number of players using losing strategies "observe" the others and switch to a winning one. This changes the dynamics of the tournament. When switching based on Fitness, deterministic strategies yield purely deterministic results. In the imitation case, stochastic behavior is introduced. Switchers are chosen at random. They also choose a strategy at random in case of a tie. Thus stochastic analysis is required to determine strategy evolution.

5.1 Simulation Tournament Imitation Dynamics

5.1.1 Description:

The goal is to simulate a full tournament for each generation. Winners are chosen based on total score accumulated. Losers are identified as non-winners. K of them then switch their strategy and join one of the winning groups. The Fitness rule isn't applied exactly. We use the total score accumulated, rather than the fraction. It shouldn't affect the validity of the process. Also note that total performance is examined, instead of individual. This causes imitators to tend to join the most populous groups, even if they use suboptimal strategies in the "one vs one" concept. It introduces an interesting evolutionary dynamic, where individuals abandon their strength to join the group with the "brighter future". All of the above is materialized through TourSimImi. It accepts as handles the payoff matrix, the strategies vector, initial population composition, number of rounds, generations and switchers. It returns the evolution of the population as a table, each dominant strategy (or strategies) per generation and a struct that indicates who switches to what strategy.

5.1.2 Pseudocode TourSimImi:

EXTRACT NUMBER OF ACTIVE STRATEGIES

INITIALIZE POPULATION, BST AND SWITCHING RECORD MATRICES

RECORD INITIAL POPULATION IN POP VECTOR

SET THE PLACEHOLDER POPULATION AS INITIAL POPULATION

```
FOR NUMBER OF GENERATIONS ITERATIONS:
    EXTRACT TOURNAMENT SCORES BY CALLING AXEL(...)
    INITIALIZE TOTAL PAYOFF TABLE
    FOR NUMBER OF ACTIVE STRATEGIES ITERATIONS:
        CREATE IDENTIFIER MASK TO OBTAIN SAME STRATEGY INDICES
        SUM TO DETERMINE THE TOTAL PAYOFF PER STRATEGY BY INDEXING THE SCORES VEC-
TOR AND STORE IT IN TOTAL PAYOFF VECTOR
    FIND WINNING STRATEGIES
    STORE IT IN BST
    INITIALIZE SWITCHING EVENTS VECTOR
    FOR NUMBER OF SWITCHERS ITERATIONS:
        SET LOSERS AS NON WINNERS
        FIND THEIR INDICES
        IF NONE: BREAK THE LOOP
        CHOOSE ONE AT RANDOM
        CHOOSE ONE OF THE WINNING STRATEGIES AT RANDOM
        STORE THE PRE-SWITCH STRATEGY
        IF THE CONVERSION RESULTS IN ACTUAL SWITCH:
            STORE THE EVENT IN THE STRUCT
            FIND OLD AND NEW STRATEGY INDICES
            UPDATE POPULATION VECTOR BY +-1 IN THE OLD AND NEW POSITIONS
            APPLY CONVERSION TO PLAYER
        RECORD IN STRUCT
        UPDATE POP TABLE
MATLAB RETURNS ALL
```

5.1.3 Experiments:

Here we shall demonstrate some experimentation. To reproduce plots shown here, run the scripts titled Imi-Sim-Fig-X from the examples folder to obtain figure number X.

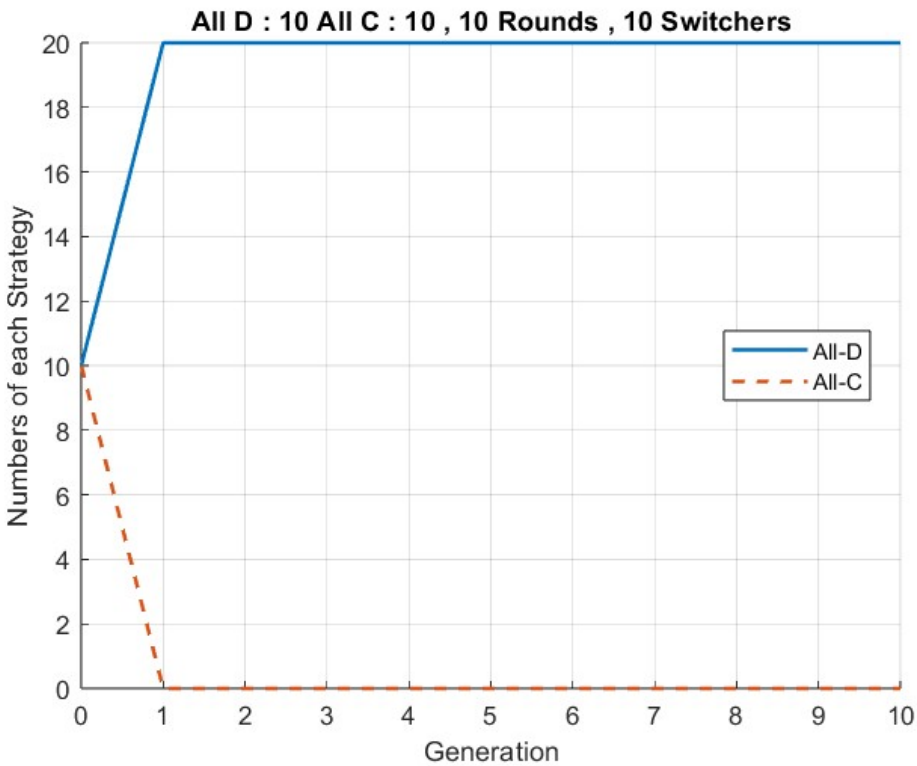


Figure 1:
Let's confirm proper functionality. We set 10 players to switch and given the population composition we expect all 10 All C players to switch to the stronger All D in one generation. Exactly as observed.

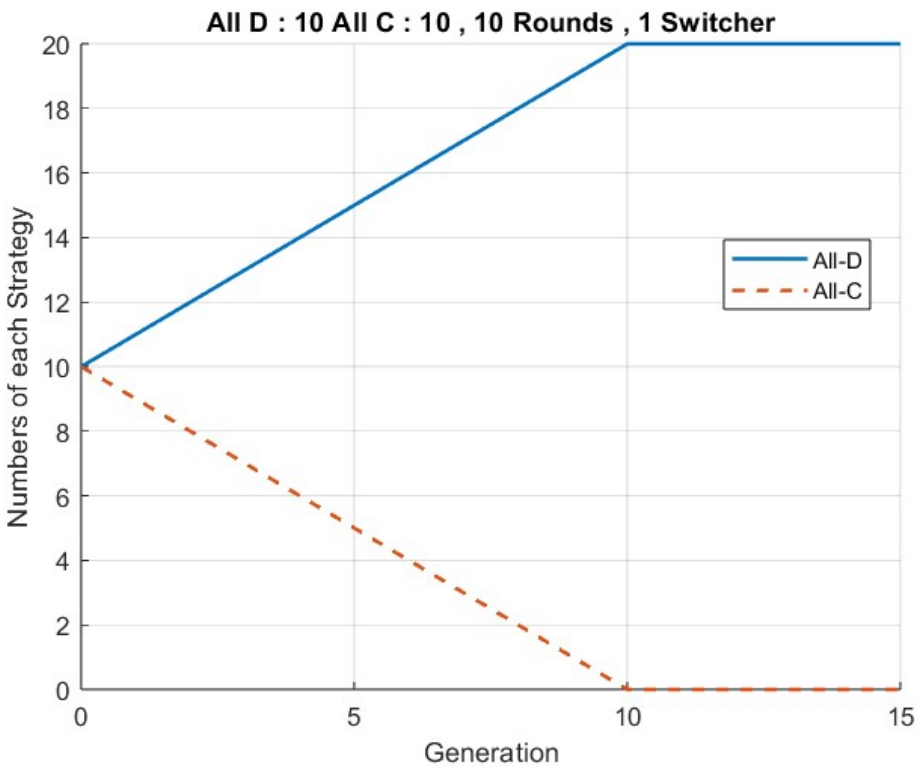
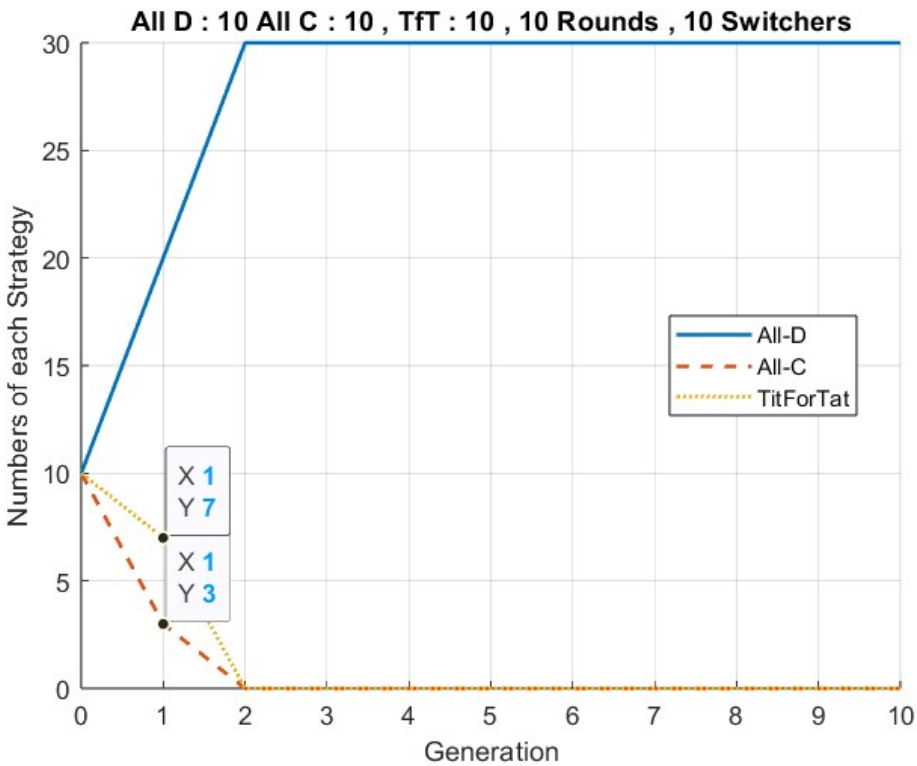
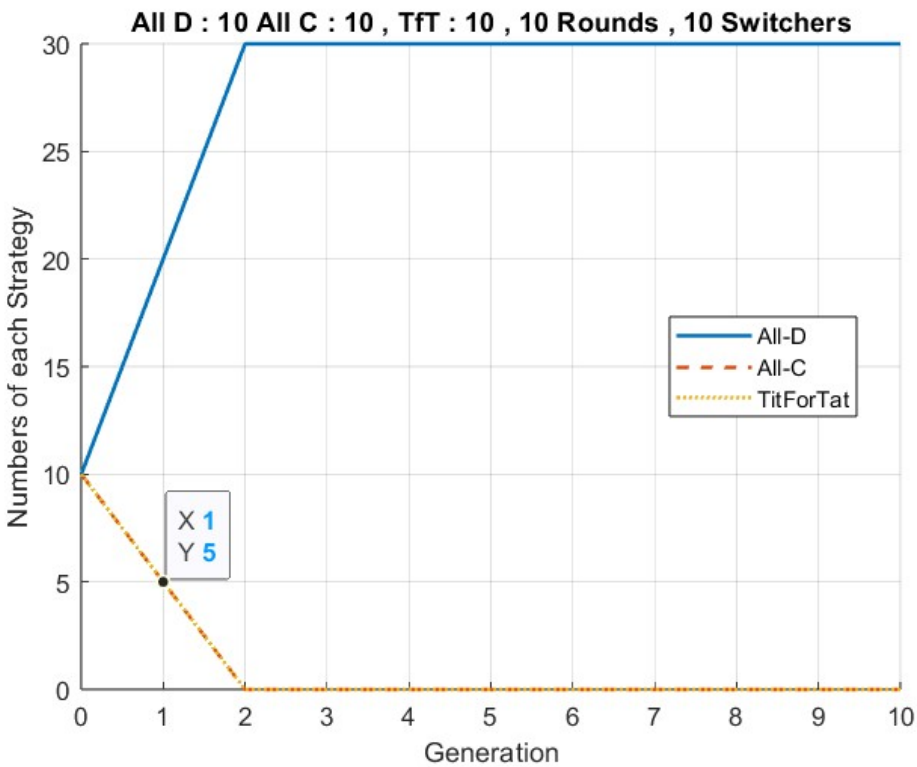


Figure 2:
Given the nature of the matchup, with one switcher we expect monotonous convergence to All D domination in 10 generations. As observed.





Figures 3 , 3.1:
We explore stochastic behavior. The above two plots are given by the execution of the same script, with the same setting. Yet we observe different convergence to the same result. This occurs because there are more than one ways to switch 10 players from the pool of the 20 losers.

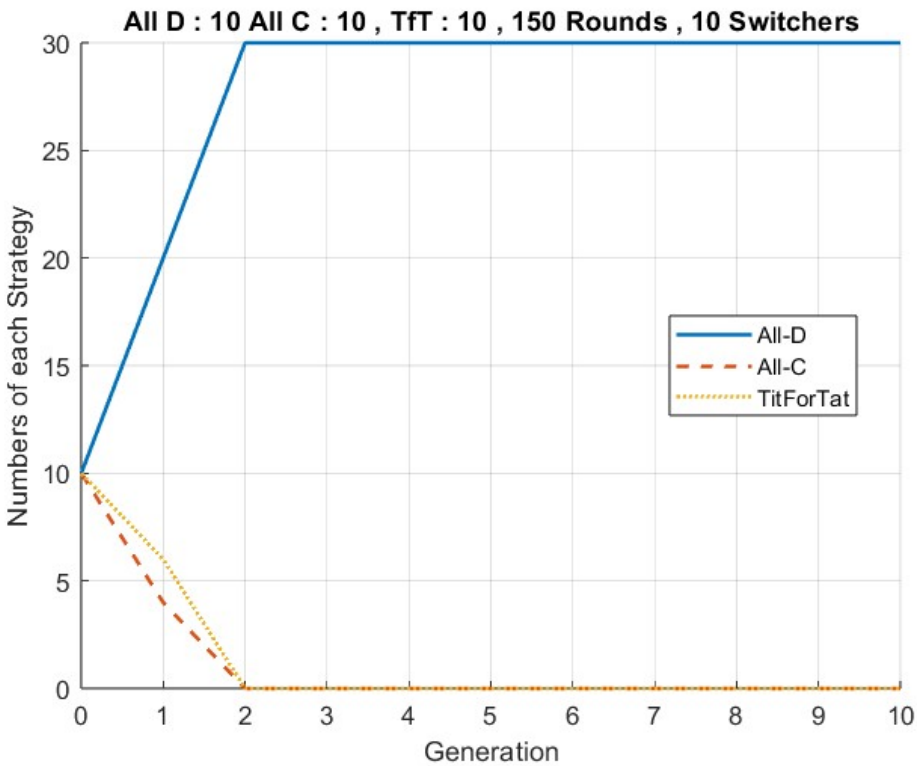


Figure 4:
Here we attempt to explore sensitivity to game length. The above setting is executed for 150-round long games, with no effect on the convergence.

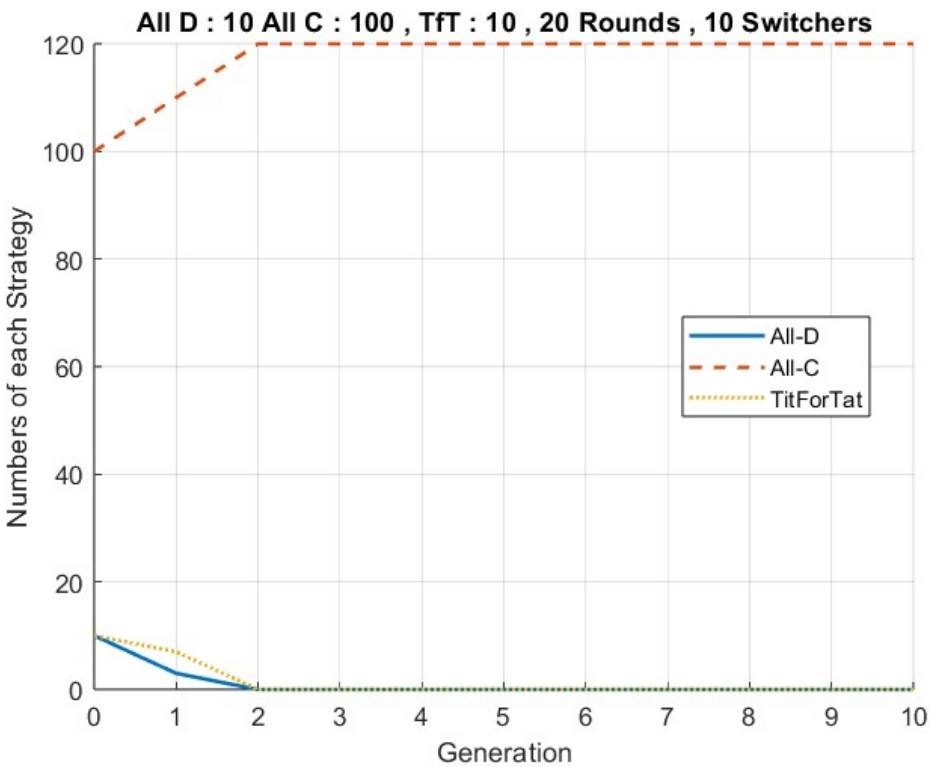


Figure 5:
Here we demonstrate the significance of population size. The weakest strategy All C dominates given a large enough population.

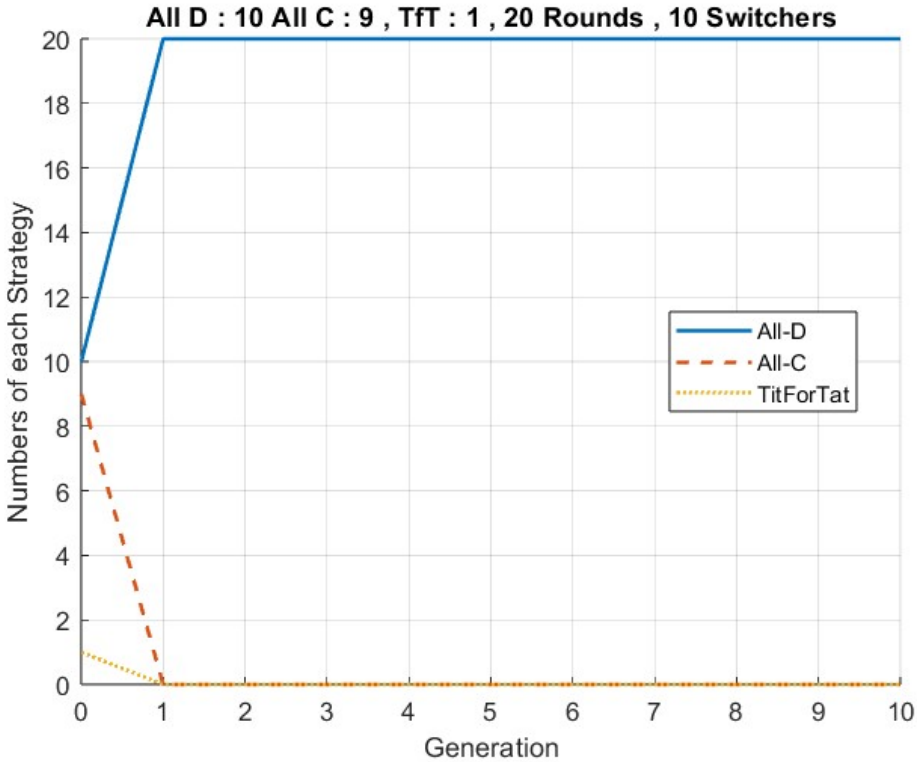


Figure 6:
Here we demonstrate how particular settings result in purely deterministic behavior. All Imi-Sim-Fig-6 executions result in the above result, since there is only one way to switch the 10 losing players at the same time.

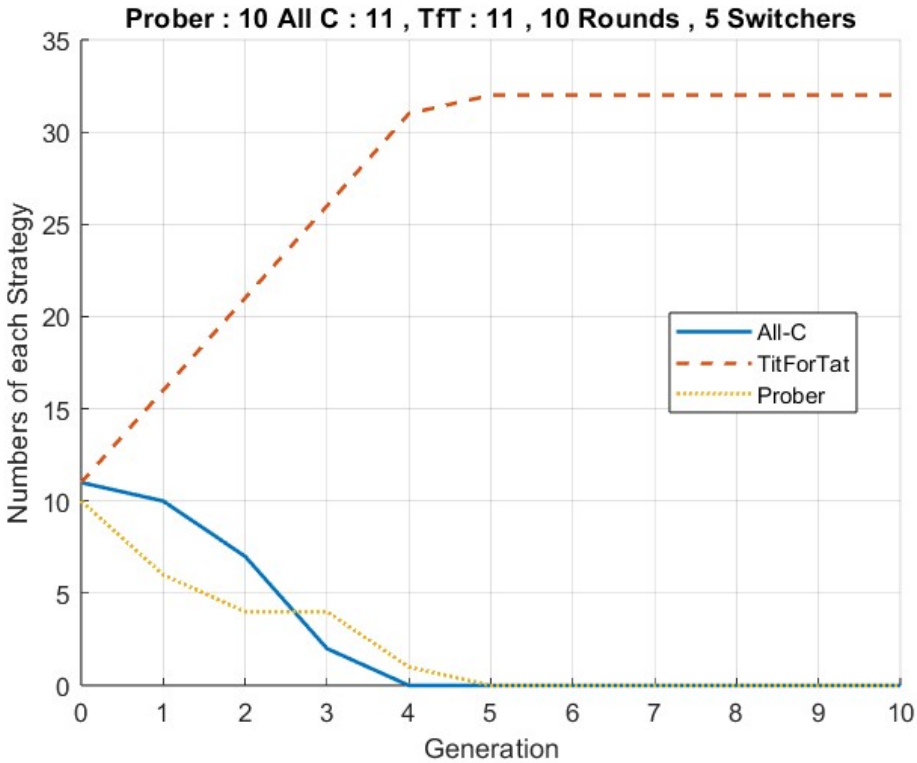


Figure 7:
Here we observe switch in population reduction rate. Prober begins with a sharper decline than All C, but later the dynamic switches with the former maintaining a slight dominance over the latter before the final elimination.

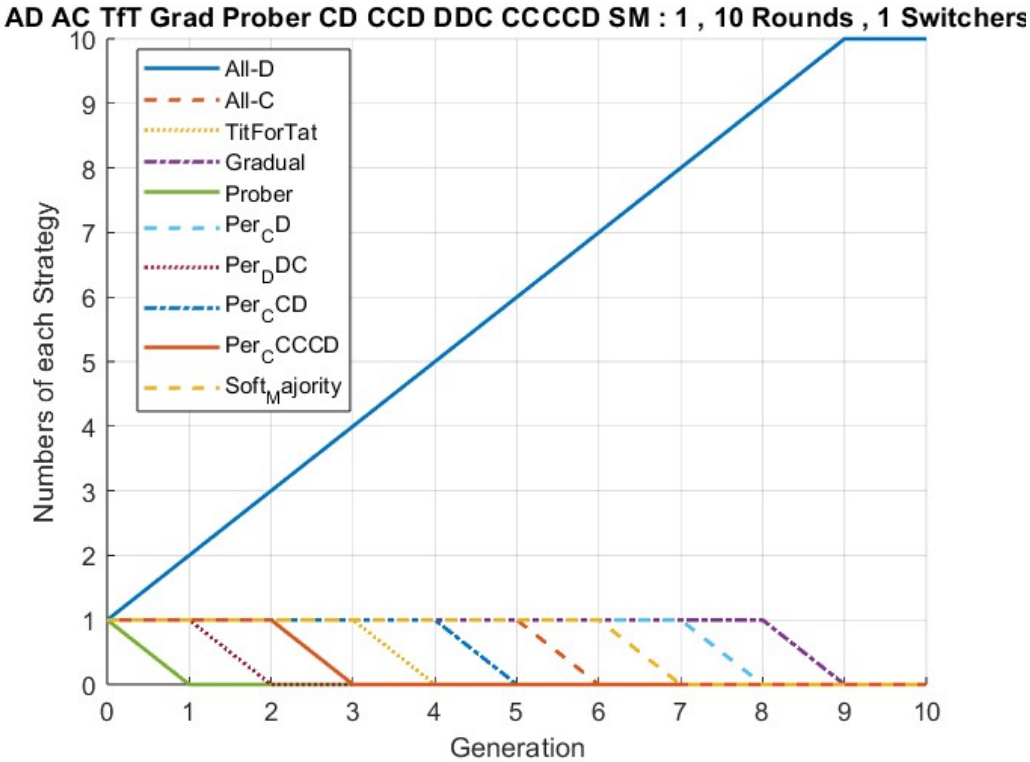
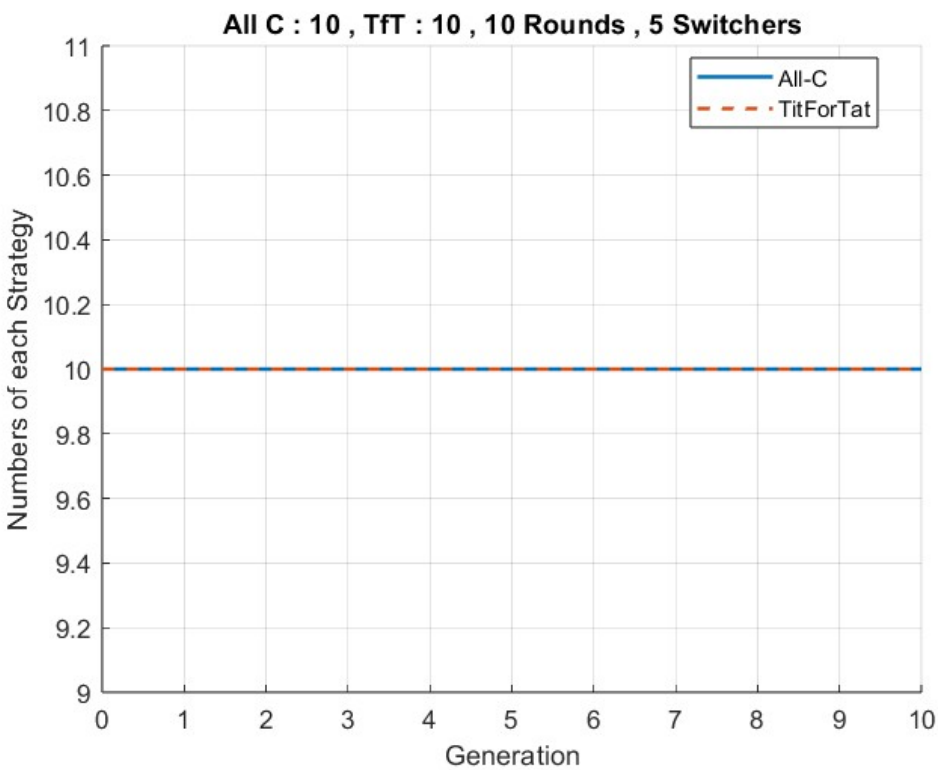
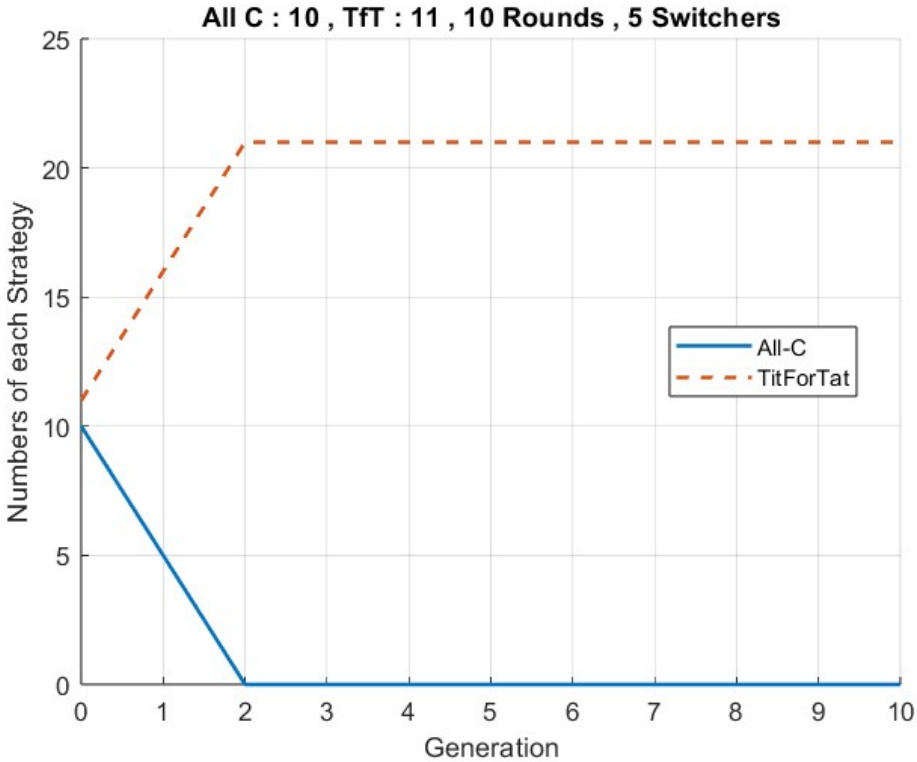


Figure 8:
Here we pit one of each strategy against each other. We observe that All D dominates, one player at a time. The other strategies move together. The stochastic element in this implementations comes down to what strategy will be eliminated and when.





Figures 9 , 9.1:
The above figures highlight the sensitivity of the outcome to the population composition, as seen before. When starting with equal number of players, All C and TfT do not exchange populations at all. Introducing one more individual sets off a chain reaction of dominance.

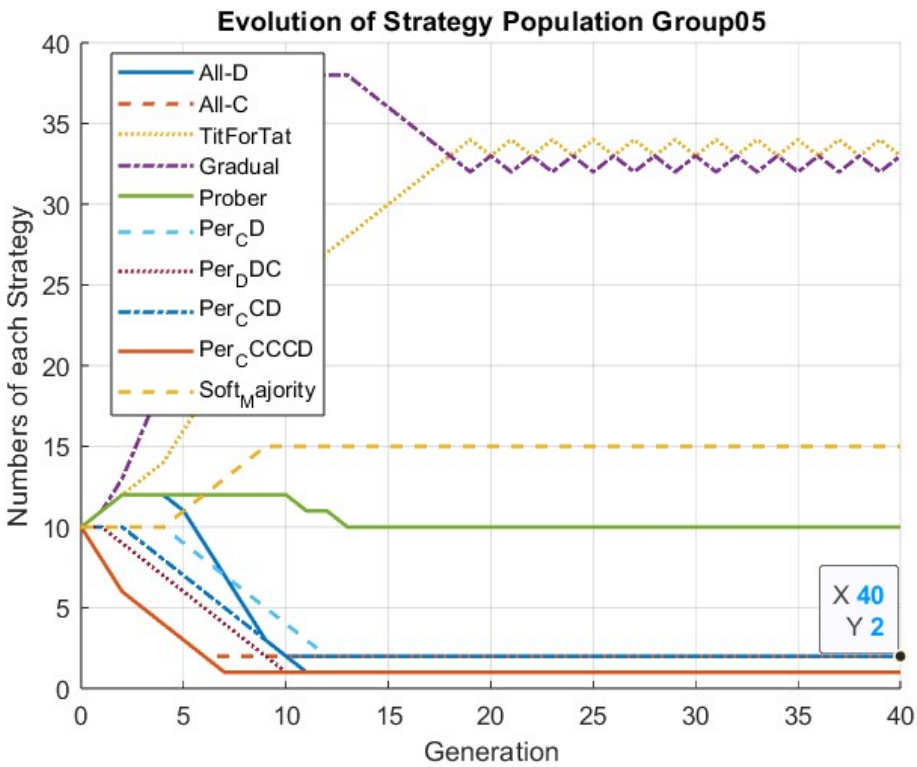


Figure 10:
This figure is borrowed from the previous report. It is not replicated by a script since we cannot find the code snippet that created it. Stochastic behavior and multiple parameters made it impossible to reproduce. How ever we observe the most interesting dynamic of all. In all of our previous examples, convergence involves elimination of most, with only one group dominating. In this instance we observe all but two strategies end up with a fixed population. The battle involves only TfT and Gradual. These two get locked in perpetual oscillatory behavior.

Once they equalize, TfT dominates and increases (by one or two). This however renders it less effective, thus prompting a population increase of Gradual.

5.2 Theoretical Analysis Of Imitation Dynamics:

The absence of simulation data forces us to perform a stochastic analysis. We can no longer model the behavior of the whole process. Uncertainty is involved. Thus our goal is to extract a Transition Matrix. It contains the probability of transitions from one strategy to another. We visualize it as a Graph Plot with certain, uncertain and infeasible transitions from one state to another. Let's walk through the analysis step by step. The walk-though isn't exactly ideal. We admit that the analysis follows the code and not vice versa. Modeling in Matlab is more intuitive and comes with the perks of easy experimentation. Even when it comes to Matlab code, however, this problem proved to be quite perplexing. Even with the Model Report given, we still lag behind.

1. For M strategies and N players we get the state vector W. Each element represents a certain group playing strategy i.

$$W \;=\; \big(W_1,\, W_2,\, \ldots,\, W_M\big), \quad W_i \in \mathbb{Z}_{\geq 0}, \quad \sum_{i=1}^{M} W_i \;=\; N.$$

The total number of different possible states is given by solving the equation for non-negative integer solutions. This results to:

$$\binom{N+M-1}{M-1}.$$

states.

2. To calculate the strategy vs strategy payoff matrix we define V(i—j) as the i vs j payoff for a given number of rounds. We then store the different strategy vs strategy match-ups in a MxM matrix by calculating

$$\sum_{j=1}^{M} W_j\, V\big(i \mid j\big).$$

3. We identify winning strategy(/ies) as those with the highest total payoff(s) accumulated. We consider as losers, therefore eligible to switch, all the non winners. Exactly like we did before.

4. The transition rules are most complex part of the analysis. The validity and success are still uncertain. K losers must be selected and assigned to the best strategy. For a more strict formulation of a loser pick, we have:

$$\ell \;=\; (\ell_1,\, \ldots,\, \ell_M)$$

$$\ell_i = 0 \quad \text{for } i \notin L^+, \qquad 0 \leq \ell_i \leq W_i \quad \text{for } i \in L^+, \qquad \sum_{i=1}^{M} \ell_i = K.$$

L is the vector containing the loser strategies. Now, picking li players from strategy i is given by:

$$\Pr(\ell) \;=\; \frac{\displaystyle\prod_{i=1}^{M} \binom{W_i}{\ell_i}}{\displaystyle\binom{n_{\text{losers}}}{K}}.$$

Now for assigning K players to dominant strategies, we define BST as the vector of best strategies, —BST— their number and a as the vector containing all possible assignment events. The probability of of assignment thus becomes:

$$\Pr(\alpha \mid \ell) \;=\; \frac{\displaystyle\binom{K}{\alpha_{i_1},\, \ldots,\, \alpha_{i_{|B|}}}}{|\text{BST}|^K}.$$

5. Time to calculate the probability of the next state transition. The thus define vector b, were:

$$b_i = \begin{cases} \alpha_i, & i \in \text{BST}, \\ 0, & i \notin \text{BST}. \end{cases}$$

Now for the next population composition we have:

$$W' = W - \ell + b$$

. Then the total transition probability from state to state' is stored in P and given by:

$$P_{s \to s'} = \sum_{(\ell, \alpha \mapsto W')} \Pr(\ell) \Pr(\alpha \mid \ell).$$

6. Absorbing states are defined as all-winner states. No switches occur and transition probability for self loop is 1 and for a new state 0.

5.2.1 Pseudocode TourTheImi:

EXTRACT NUMBER OF PARTICIPATING STRATEGIES

CALCULATE TOTAL POPULATION

GET STATES MATRIX BY CALLING GET ALL STATES
it is a matrix of height equal to the total number of different states and width equal to the number of strategies

EXTRACT THE NUMBER OF DIFFERENT STRATEGIES

INITIALIZE STRATEGY VS STRATEGY PAYOFF MATRIX

FOR NUMBER OF STRATEGIES ITERATIONS:

    FOR NUMBER OF STRATEGIES ITERATIONS:

        CALCULATE RESULT BY CALLING MATCH 2

        STORE IT IN CURRECT POSITION

INITIALIZE TRANSITION MATRIX P

FOR NUMBER OF DIFFERENT STATES ITERATIONS:

    DEFINE W AS CURRENT STATE

    W is the population composition for the given state

    CALCULATE PAYOFF VECTOR FOR STRATEGY MATCH-UPS

    FIND WINNING STRATEGIES

    IDENTIFY LOSERS AS NON WINNERS

    ENSURE POSITIVE POPULATIONS

    CALCULATE NUMBER OF LOSERS

    PRINT DEBUGGING INFORMATION

    IF NO LOSERS:

        PRINT INFO

        SET AS ABSORBING STATE

CALCULATE TOTAL WAYS TO PICK K LOSERS IN MATRIX FORM

CALCULATE TOTAL WAYS TO ASSIGN TO STRATEGIES

FOR NUMBER OF DIFFERENT PICK COMBINATIONS ITERATIONS: EXTRACT PICK VECTOR AS CURRENT ROW OF TOTAL PICK MATRIX

CALCULATE THE TOTAL NUMBER OF WAYS TO CHOOSE K OUT OF NUMBER OF LOSERS

CALCULATE PROBABILITY FOR GIVEN PICK

FOR NUMBER OF WAYS TO PICK FROM DIFFERENT STRATEGIES:

EXTRACT STRATEGY DISTRIBUTION FROM THE CORRESPONDING ROW STRAT PICKS MATRIX

INITIALIZE STRATEGY PCIKS VECTORS AS ZEROS

ASSIGN SAID DISTRIBUTION TO WINNING POSITIONS

EXTRACT THE TOTAL NUMBER OF WAYS TO ASSIGN SWITCHERS TO THE STRATEGIES

CONVERT TO PROBABILITY

CALCULATE THE NEXT STATES POPULATION COMPOSITION

FIND THE INDEX OF DIFFERENT STATES THAT CORRESPONDS TO CURRENT COMPOSITION

UPDATE THE TRANSITION MATRIX

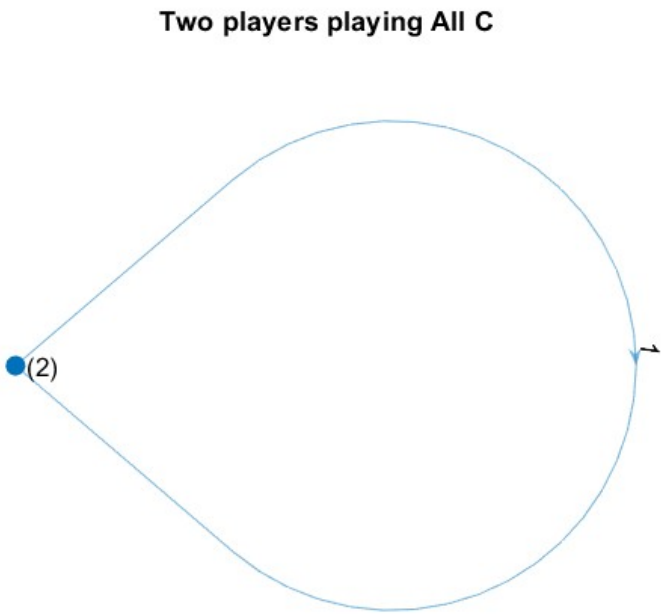
LOCATE AND CONVERT ZERO ROWS TO ABSORBING STATES

MATLAB RETURNS P

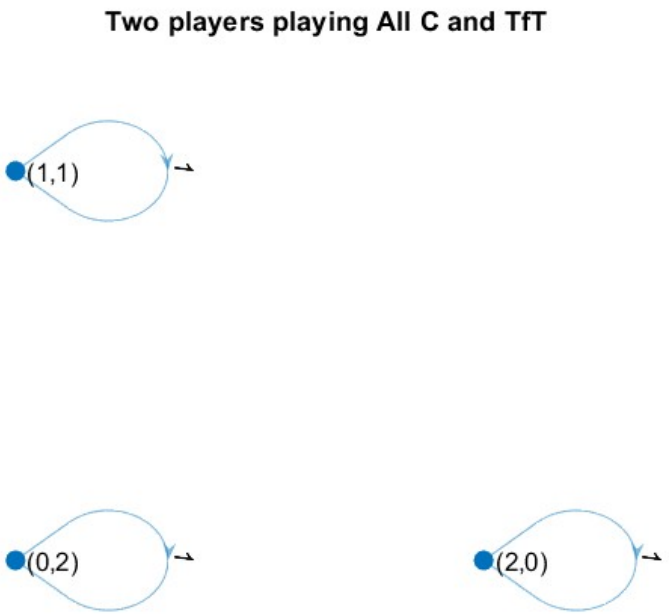
5.2.2 Results:

Here we demonstrate the results to extract information about code success.

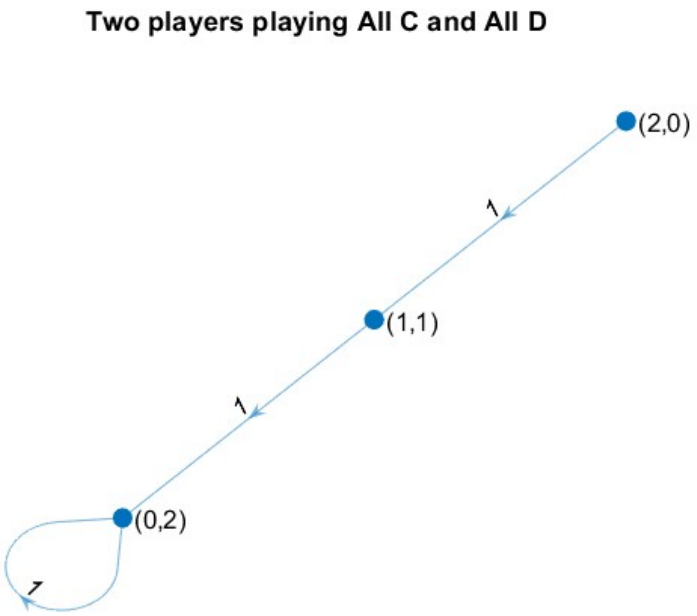
Only one player should yield an absorbing state. It does indeed.



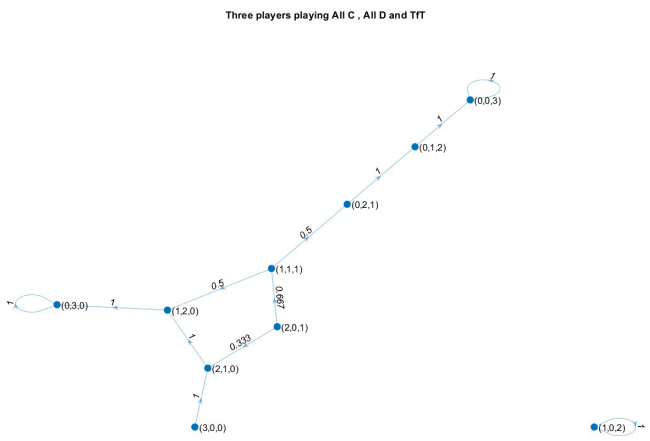
Two players playing the same strategy should also be an absorbing state. It is indeed.



Two players playing equivalent strategies should also result to three distinct absorbing states. It does indeed.



Two players playing All D and All C should transition from one All D and one All C to two playing All D with a probability of 1. Which indeed happens. But here the shortcomings of our code begin to show. All C vs All C should be a sinkhole. Yet it isn't. As if the algorithm knows that All D is better, although no All D players are present in the matchup. We failed to locate how and why. Under that assumption, this transition if canonical.



In the 1 All C, 1 All D and 1 TfT matchup we observe the same paradox. Three All C players transition to All D step by step. As if they know it performs better. The rest of the structure makes sense overall.

6 References:

1) Alexander, J. McKenzie. Evolutionary game theory. Cambridge University Press, 2023.
2) Axelrod, Robert, and William D. Hamilton. "The evolution of cooperation." Science, Vol. 211, No. 4489, pp. 1390-1396, 1981.
3) Axelrod, Robert, and Douglas Dion. "The further evolution of cooperation." Science, Vol. 242, No. 4884, pp. 1385-1390, 1988.
4) Mathieu, Philippe, Bruno Beaufils, and Jean-Paul Delahaye. "Studies on Dynamics in the Classical Iterated Prisoner's Dilemma with Few Strategies" European conference on artificial evolution. Springer: Berlin Heidelberg, 1999.